ZOMBI2 performance

Scaling of species-tree and gene-family simulation

Performance analysis

generated for the ZOMBI2 project

2026-07-04

Abstract

We benchmark the two core operations of ZOMBI2 — simulating a species tree and evolving gene families along it — across five orders of magnitude in tree size, on a single laptop, and set them against the legacy ZOMBI 1. Both operations are essentially linear in the number of tips: a **three-million-tip species tree** is simulated in **≈18 s using ≈2 GB**, and **one-million-tip** gene-family copy-number profiles in **≈18 s using ≈3.2 GB**. Gene families are emitted in one of three modes — a full per-event log, a compact *event trace* (from which gene trees stay reconstructable), and a counts-only profile. The trace and profile paths scale to a million tips, whereas the full log tops out near 100,000 because it materialises one Python object per event. The dense families × species profile matrix, quadratic in tip count, was the one component that did not scale; a sparse (COO) representation moved its ceiling from ∼100,000 to several million. Replicate-level parallelism gives a 3.9× speed-up on a 10-core chip. On the identical gene-family task, ZOMBI2 runs **over 1000× faster than ZOMBI 1** and keeps scaling far past ZOMBI 1's ∼1,200-tip practical ceiling. All raw timings, the code that produced them, and these figures are reproducible from the `performance_analysis/` workspace.

1 Setup and methodology

Workload. Every benchmark uses one fixed regime so the numbers stay comparable: a birth–death species model BirthDeath(λ=1.0, μ=0.3) of crown age 2.0, and gene families evolving under uniform per-copy rates (D=0.2, T=0.1, L=0.25, O=0.5; 20 founding families). Loss exceeds duplication, so family sizes never run away. The built-in model runs on the compiled `zombi2_core` (Rust) engine, which `simulate_genomes` selects automatically.

Measurement. Wall-clock time is taken with a monotonic clock (`time.perf_counter`) after one untimed warm-up; each point is repeated several times and we report the *median* with a shaded inter-quartile (25–75%) band. Each timed repeat runs with the cyclic garbage collector *disabled* (with a full `gc.collect()` between repeats), exactly as `timeit` does: without this the object-heavy full-log path swings wildly as a collection fires at a different point each run: this isolates intrinsic cost from the collector's schedule. Species trees are built once per size and reused across the gene-family series, so those comparisons are a fair A/B on identical inputs; only the timed call sits inside the timer. Peak memory (resident set size) is measured in an *isolated subprocess* per size — the only way to get a clean per-size figure from a high-water-mark counter — and there the collector is left *on*, so the memory numbers reflect a normal run.

Machine. Apple Silicon, 10 cores (4 performance + 6 efficiency), 34 GB RAM, macOS; CPython 3.12, ZOMBI2 `0.2.0.dev0`. Absolute numbers are machine-specific; the *scaling* is not.

2 Species-tree simulation scales to millions of tips

Simulating the species tree is $O(N)$ in the number of tips and featherweight (Figure 1). The backward reconstructed sampler — the default, which conditions on N extant tips — takes 50 ms at 10,000 tips, 0.55 s at 100,000, 5.7 s at 1,000,000, and 17.6 s at 3,000,000; the linear trend extrapolates cleanly to tens of millions (`python run.py --full`). The forward complete-tree sampler is comparable up to ∼100,000 tips but grows heavier beyond, because it materialises every extinct lineage too (a 10^6-tip *extant* tree carries $\approx 1.4 \times 10^6$ total nodes).

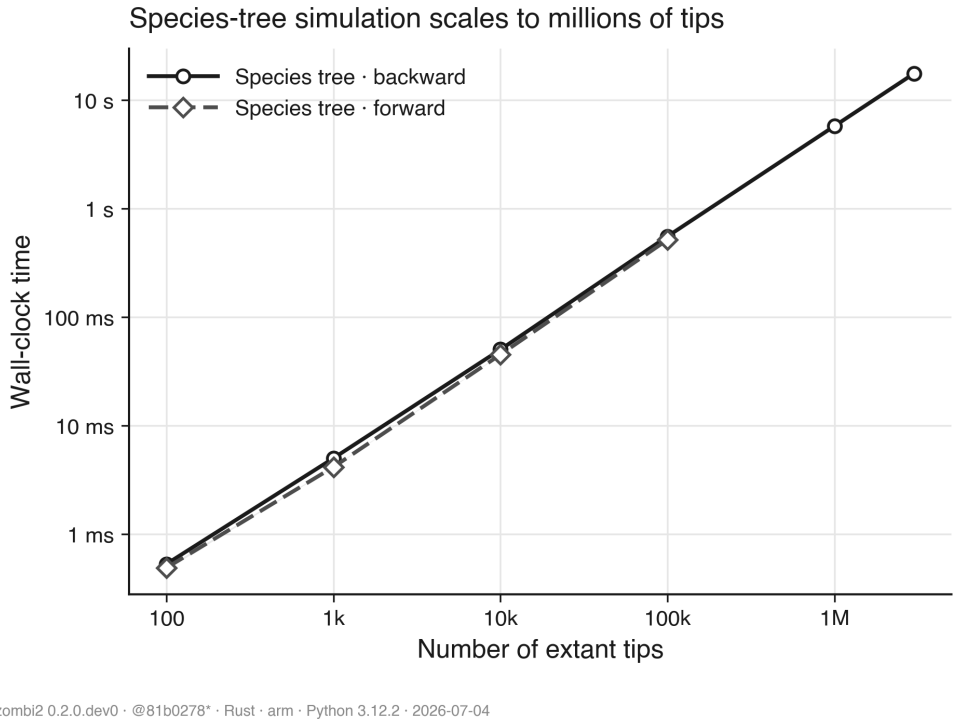

Figure 1. Wall-clock time to simulate a species tree versus number of extant tips (log–log). Both samplers track the $\propto N$ guide. Median of repeated runs; band is the inter-quartile range.

3 Gene-family simulation: three output modes

Evolving D/T/L/O gene families along a fixed tree offers three outputs of increasing richness (Figure 2). The *full event log* (`output="genomes"`) records every event as a Python object, so its cost tracks the event count and it is practical to ∼100,000 tips (4.9 s, ≈3,000,000 events). The *event trace* (`output="trace"`) records the same D/T/L/O history but drops the speciation "carry" events entirely — a gene keeps its id across speciations, and the implied bifurcations are recovered by replaying the trace against the species tree — so gene trees remain reconstructable on demand while the cost falls to barely above counts-only: 1.4 s at 100,000 tips and 19 s at 1,000,000. The *counts-only* profile (`output="profiles"`) — the presence/absence dataset used for approximate Bayesian computation and downstream analyses — is the cheapest and, after the sparse-output change (§4), scales linearly into the millions: 63 ms at 10,000 tips, 1.1 s at 100,000, and 16 s at 1,000,000 tips.

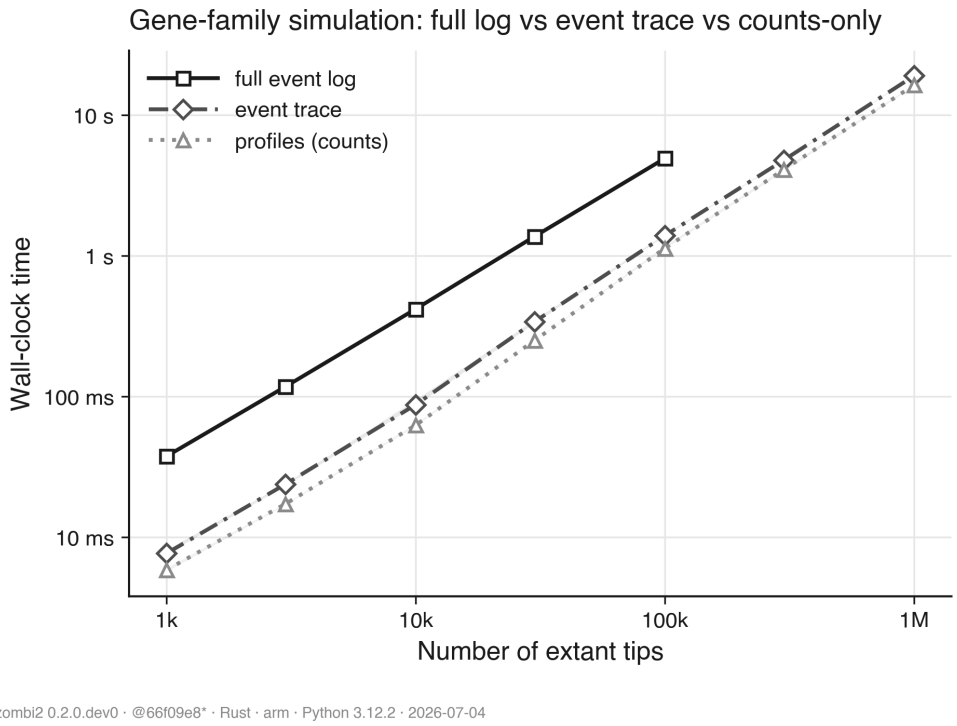

Figure 2. Gene-family simulation versus tip count (log–log). The event trace and counts-only profile both run to 1,000,000 tips along the $\propto N$ guide; the full event log is heavier and caps near 100,000 tips.

4 Memory footprint and the sparse-output change

Peak memory (Figure 3) tells the "how big can we go" story. The species tree is the lightest, roughly 0.7 GB per million tips (0.13 GB at 100,000, 0.71 GB at 1,000,000, 2.0 GB at 3,000,000).

The copy-number *profile* was originally a dense families × species array. Because the number of families grows in proportion to the number of species, that array is $O(N^2)$ — it would demand ≈25 GB of address space already at 100,000 tips and ≈8 TB at 1,000,000, a hard wall well before the simulation itself struggled. We replaced it with a **sparse (COO) representation** storing only the present cells, which is $O(N)$: the profile now costs 0.43 GB at 100,000 tips and 3.2 GB at 1,000,000 — sizes the dense form could never reach. The *event trace* sits just above it: by dropping the redundant speciation "carry" rows (recoverable from the species tree) and keeping only per-family leaf counts, it holds the whole D/T/L/O history for a million tips in 5.1 GB — close to the counts-only floor, and far below the full log's 3.5 GB already at 100,000 tips (one Python object per event). The residual gap to the profile is the trace columns themselves, held as Python lists.

5 Replicate-level parallelism

Independent replicates are embarrassingly parallel. Running 20 independent full simulations of 10,000-tip trees through a process pool (Figure 4) drops the wall-clock from 21.4 s on one worker to 5.4 s on ten — a 3.9× speed-up. The curve bends away from the ideal beyond four workers: this chip has only four performance cores (the other six are slower efficiency cores), and on macOS each worker re-imports the library under the `spawn` start method. On a homogeneous, `fork`-based cluster (e.g. Linux/SLURM) the same workload scales closer to ideal.

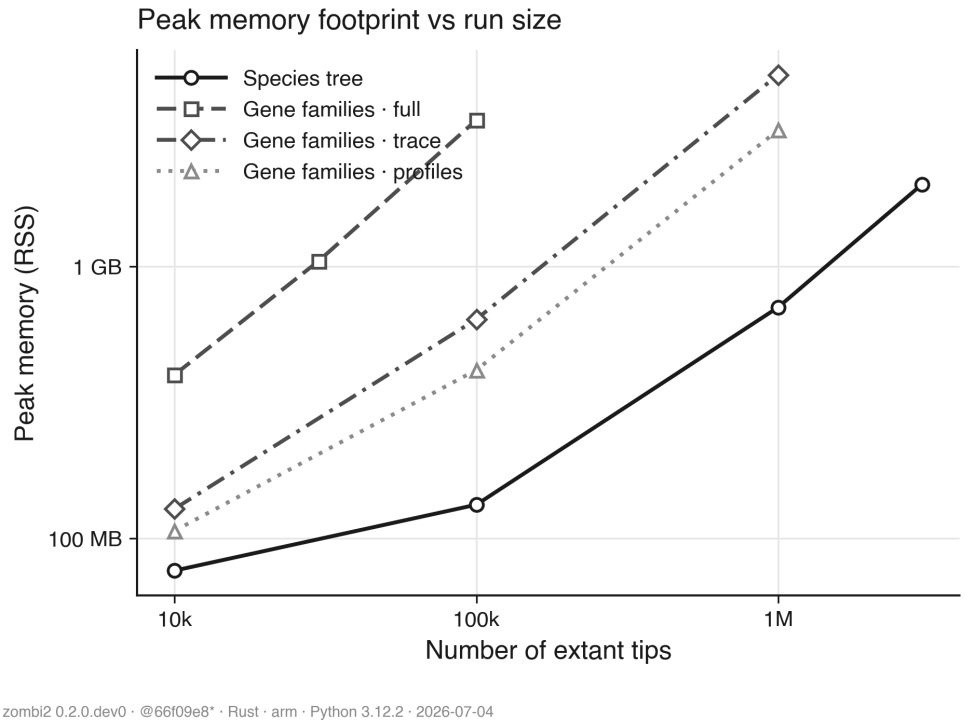

Figure 3. Peak resident memory versus tip count (isolated subprocess per size). The species tree, the sparse profiles, and the event trace all scale linearly into the millions (the trace just above the counts-only floor); the full event log is the memory-heaviest path and caps near 100,000 tips.

6 Comparison with ZOMBI 1

To place the Rust engine in context we ran the *same* gene-family task on the legacy ZOMBI 1 (pure Python): a pure-birth Yule tree (λ=1) grown to a matched extant-tip count, the same rate regime (D=0.2, T=0.1, L=0.25, O=0.5; 20 founding families; size-neutral replacement transfers), timing only the genome step (ZOMBI 1's `Gm` mode) in both. The two tools produce comparable family counts, confirming a like-for-like workload (Figure 5). ZOMBI 1 is strongly super-linear and reaches a **practical ceiling near 1,200 tips** (≈55 s); ZOMBI2 runs the same 1,000-tip simulation in 38 ms versus ZOMBI 1's 48.2 s — **over** 1000× **faster** — and continues smoothly to 100,000 tips and beyond, where ZOMBI 1 is long since infeasible.

7 Summary

Table 1. Headline results at representative scales (median wall-clock; peak RSS from the isolated-subprocess memory runs).

Task	Tips N	Time	Unit	Peak memory
Species tree (backward)	1,000,000	5.7	s	0.71 GB
Species tree (backward)	3,000,000	17.6	s	2.0 GB
Gene families, full log	100,000	4.9	s	3.5 GB
Gene families, event trace	100,000	1.4	s	0.64 GB
Gene families, event trace	1,000,000	19.0	s	5.1 GB
Gene families, profiles	1,000,000	16.0	s	3.2 GB

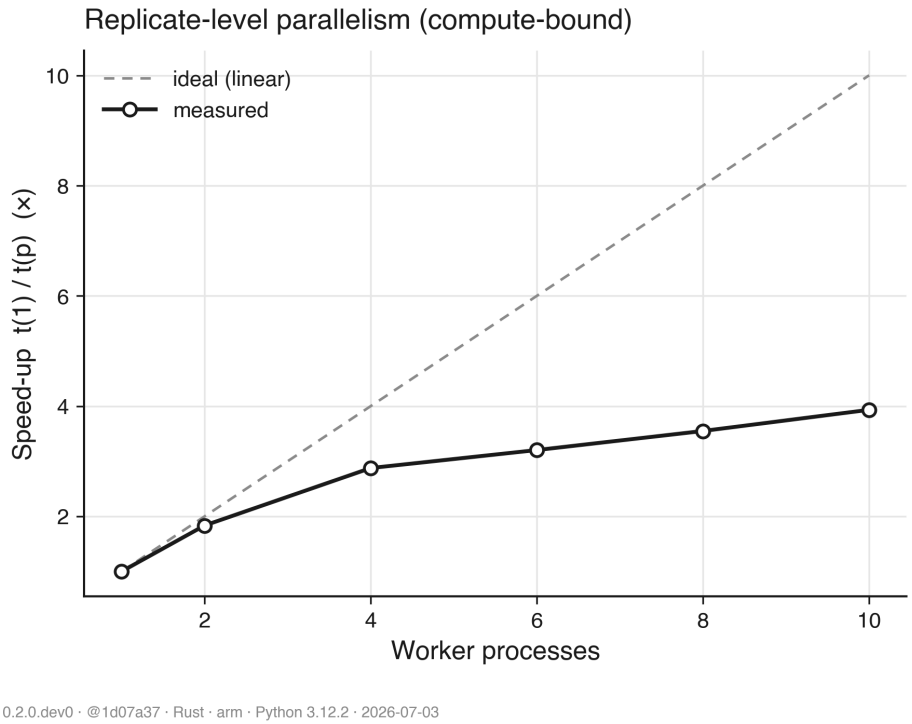

Figure 4. Speed-up of a fixed batch of independent simulations versus worker processes, against the ideal-linear reference. The plateau past four workers reflects the 4-performance/6-efficiency-core split and `spawn` overhead.

Bottom line. Both core operations are linear in tip count and run at million-tip scale on a laptop; the species tree reaches the millions with room to spare. The only super-linear component was the dense profile output, now sparse and linear. Of the three gene-family outputs, the counts-only profile is the cheapest, the event trace keeps gene trees reconstructable for barely more (both reach a million tips at close to the same time and memory), and the full per-event log — the richest but heaviest — is best kept to ∼100,000 tips. Against the legacy ZOMBI 1 the Rust engine is over three orders of magnitude faster on the same task (§6). An overview of the headline results is shown in Figure 6.

Reproducible from `performance_analysis/`: `python run.py` writes raw timings to `results/*.json`; `python plot.py` renders the figures. Each result file embeds its own provenance (git commit, interpreter, CPU).

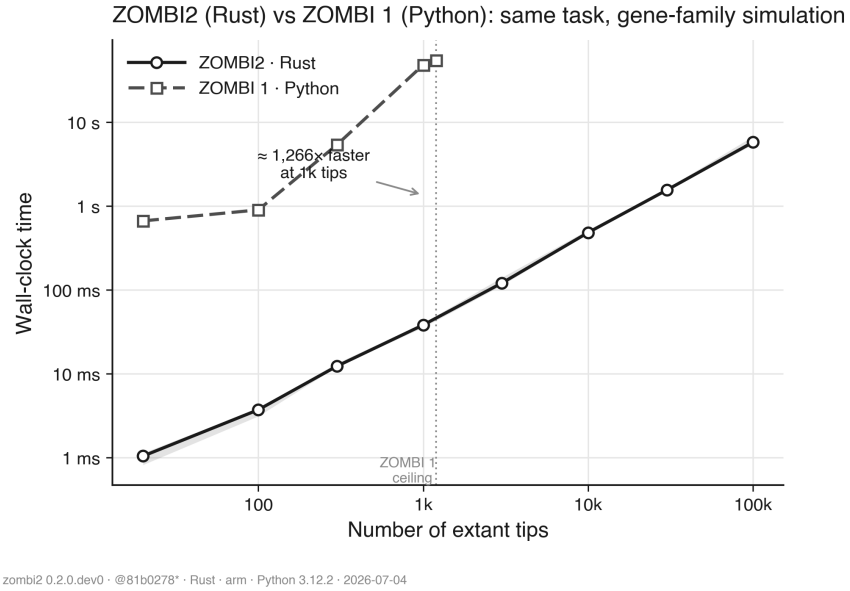

Figure 5. ZOMBI2 (Rust) versus ZOMBI 1 (legacy Python) on the same gene-family task, matched tip count and rate regime (log–log). ZOMBI2 is over three orders of magnitude faster; the dotted line marks ZOMBI 1's practical ceiling. Opt-in benchmark (`vs_zombi1`); see `vs_zombi1/NOTES.md`.

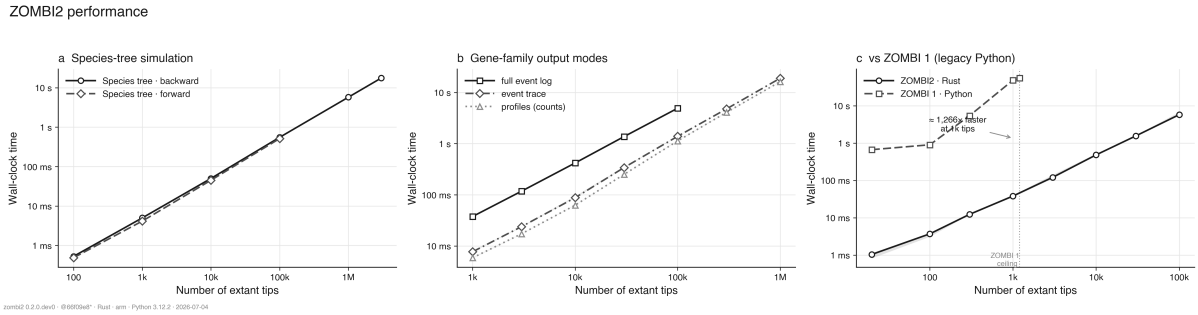

Figure 6. Overview: (a) species-tree scaling, (b) the three gene-family output modes, (c) ZOMBI2 versus the legacy ZOMBI 1 on the same task.